Simulation of Quantum Particle in a One-Dimensional Potential Box using C Programming

By

Akshat Tyagi(241031016)

Anshit Guleria(241032004)

Aansh Jain(241030210)

Toshit kaul(241030436)

Saubhagya Sharma()

UNDER THE SUPERVISION OF
Name of Supervisor : Prof.Dr. P.B.Barman

Department of Computer Science & Engineering and Information Technology

Jaypee University of Information Technology, Waknaghat,
173234, Himachal Pradesh

1.1 Introduction

Quantum mechanics plays a vital role in modern physics, offering deep insights into how matter behaves at atomic and subatomic scales. One of the most basic — yet powerful — models used to explain quantum behavior is the "particle in a box." This model imagines a particle that can move freely within a confined space but cannot escape its boundaries. While it’s a simplified scenario, it helps us understand key concepts like quantized energy levels, boundary conditions, and the shape of wavefunctions.

In this project, we focus on simulating a particle trapped in a one-dimensional infinite potential well using the C programming language. The goal is to compute and visualize the particle’s energy levels and wavefunctions. Through this simulation, we aim to bring quantum theory to life and better understand how particles behave when they’re confined at such small scales.

1.2 Objective

This project was designed with a few key goals in mind:

- To simulate a one-dimensional quantum system using a simple C program.
- To calculate and display the discrete energy levels of a particle confined in a box.
- To evaluate the wavefunctions for different quantum states and show how they change with position.
- To give students a hands-on way to understand quantum concepts that are often only discussed theoretically.

By achieving these goals, we aim to make quantum mechanics more accessible and intuitive through computation.

1.3 Motivation

Learning quantum mechanics can be challenging, especially when everything is taught through equations and abstract theory. Concepts like energy quantization and wavefunctions can seem distant and difficult to visualize. This project was motivated by the desire to make those ideas more concrete.

By writing a program that actually calculates and displays these values, we can see how theory translates into numbers and patterns. It’s a great way to deepen our understanding, and it also opens the door to more advanced simulations — whether in physics, AI, or nanotechnology. Understanding these basics is a stepping stone toward exploring the quantum world more confidently.

1.4 Project Methodology

This simulation was developed using the C programming language because it's fast, efficient, and gives precise control over calculations. Here's a breakdown of how the project was approached:

1.4.1 Mathematical Modeling

The simulation is based on solving the time-independent Schrödinger equation for a particle in an infinite potential well:

- **Energy Levels:**
  $E_n = (n^2 \cdot h^2) / (8 \cdot m \cdot L^2)$

  where:

  - En= energy of the nth quantum state
  - h = Planck's constant
  - m = mass of the particle (assumed to be an electron)
  - L = length of the box
- **Wavefunction:**
  $\psi_n(x) = \sqrt{(2 / L)} \cdot \sin(n \cdot \pi \cdot x / L)$
- This function gives the spatial probability amplitude for the particle.

1.4.2 Code Design and Features

- **Input**: The program starts by asking for the length of the box and the unit (micrometers, nanometers, or meters). It also takes in the highest quantum number and the number of steps to use when calculating wavefunctions.
- **Conversion**: The entered unit is converted to meters to maintain consistency.
- **Computation**: The program calculates energy levels for all quantum states up to the entered maximum. It also calculates wavefunction values at evenly spaced points across the box.
- **Output**: Results are printed in scientific notation, showing both energy levels and wavefunction probabilities.

Efficiency and Accuracy

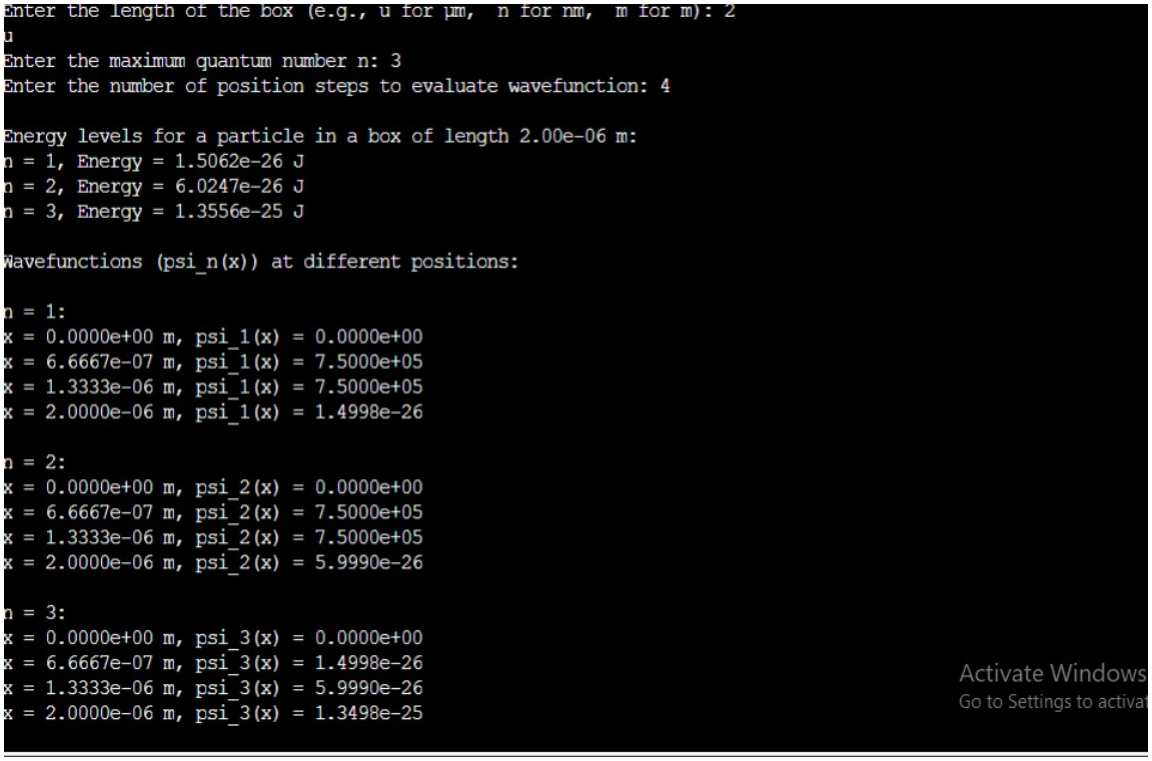
All mathematical operations use `double` precision, and built-in functions like `pow()` and `sin()` ensure accurate results. The code avoids unnecessary calculations by keeping loops efficient and separating concerns like input validation and unit conversion.

```
#include <stdio.h>
#include <math.h>

#define PLANCK 6.62607015e-34
#define MASS 9.10938356e-31
#define PI 3.141592653589793

double energy_level(int n, double L) {
    return (pow(n, 2) * pow(PLANCK, 2)) / (8.0 * MASS * pow(L, 2));
}

double wavefunction(int n, double x, double L) {
    return sqrt(2.0 / L) * sin(n * PI * x / L);
}

int main() {
    int n, max_n, i, steps;
    double L, x, dx;

    char unit;
printf("Enter the length of the box (e.g., u for µm,  n for nm,  m for m):
");
scanf("%lf %c", &L, &unit);

if (unit == 'u') L *= 1e-6;
else if (unit == 'n') L *= 1e-9;
else if (unit == 'm') L *= 1.0;
```

```
else {
    printf("Unknown unit. Assuming meters.\n");
}

    printf("Enter the maximum quantum number n: ");
    scanf("%d", &max_n);

    printf("Enter the number of position steps to evaluate wavefunction:
");
    scanf("%d", &steps);

    dx = L / (steps - 1);

    printf("\nEnergy levels for a particle in a box of length %.2e m:\n", L);
    for (n = 1; n <= max_n; n++) {
        double E = energy_level(n, L);
        printf("n = %d, Energy = %.4e J\n", n, E);
    }

    printf("\nWavefunctions (psi_n(x)) at different positions:\n");
    for (n = 1; n <= max_n; n++) {
        printf("\nn = %d:\n", n);
        for (i = 0; i < steps; i++) {
            x = i * dx;
            printf("x = %.4e m, psi_%d(x) = %.4e\n", x, n,
pow(wavefunction(n, x, L),2));
        }
    }
```

```
    return 0;
}
```

Sample Output:



1.5 Deliverables / Outcomes

Here’s what the project delivered:

- A fully functional C program that takes input from users and performs precise quantum calculations.
- Computed energy levels for multiple quantum states, displayed clearly.
- Wavefunctions calculated across the box, helping visualize how the particle’s probability density changes.
- A better understanding of quantum behavior, especially how boundary conditions shape particle motion.
- A modular code structure that could easily be expanded to include plotting or higher-dimensional systems in the future.

Conclusion

This project effectively demonstrates how a simple yet fundamental quantum mechanical system can be simulated using basic C programming. By computing

energy levels and wavefunctions for a particle in a box, we bring abstract quantum principles to life in a numerical format.

The outcomes highlight the elegance of quantum systems: energy quantization, boundary-induced wave patterns, and probability distributions that vary with quantum state. This lays the foundation for simulating more complex systems and inspires deeper exploration into computational physics.

In future work, the program could be enhanced to plot wavefunctions using graphical libraries like GNUplot or integrated with Python for visual output. Additionally, support for time-dependent solutions or multi-particle systems could further elevate its educational value.

References

1. Griffiths, D. J. (2018). *Introduction to Quantum Mechanics* (2nd ed.). Pearson Education.

2. Resnick, R., & Eisberg, R. (2002). *Quantum Physics of Atoms, Molecules, Solids, Nuclei, and Particles*. John Wiley & Sons.

3. NIST. (2023). Planck Constant. *National Institute of Standards and Technology*. Retrieved from https://physics.nist.gov

4. Cplusplus.com. (2024). *Math.h Library Reference*. Retrieved from https://cplusplus.com/reference/cmath/

5. Beiser, A. (2003). *Concepts of Modern Physics*. McGraw-Hill Education.